

Programação Web

Aula 4: CSS – Layouts Modernos

Thales Teles



Objetivos de Aprendizagem

Ao final da aula, você será capaz de:

- Compreender em profundidade o **CSS Box Model** (`content`, `padding`, `border`, `margin`).
- Entender o problema do cálculo de dimensões e a importância do `box-sizing: border-box`.
- Introduzir o **Flexbox** como a principal ferramenta para criação de layouts unidimensionais (linhas ou colunas).



Objetivos de Aprendizagem

Ao final da aula, você será capaz de:

- Dominar as propriedades essenciais do Flex Container (`display: flex`, `flex-direction`, `justify-content`, `align-items`).
- Aplicar Flexbox para resolver problemas reais de layout: criar uma barra de navegação horizontal e um layout de colunas (conteúdo + sidebar).

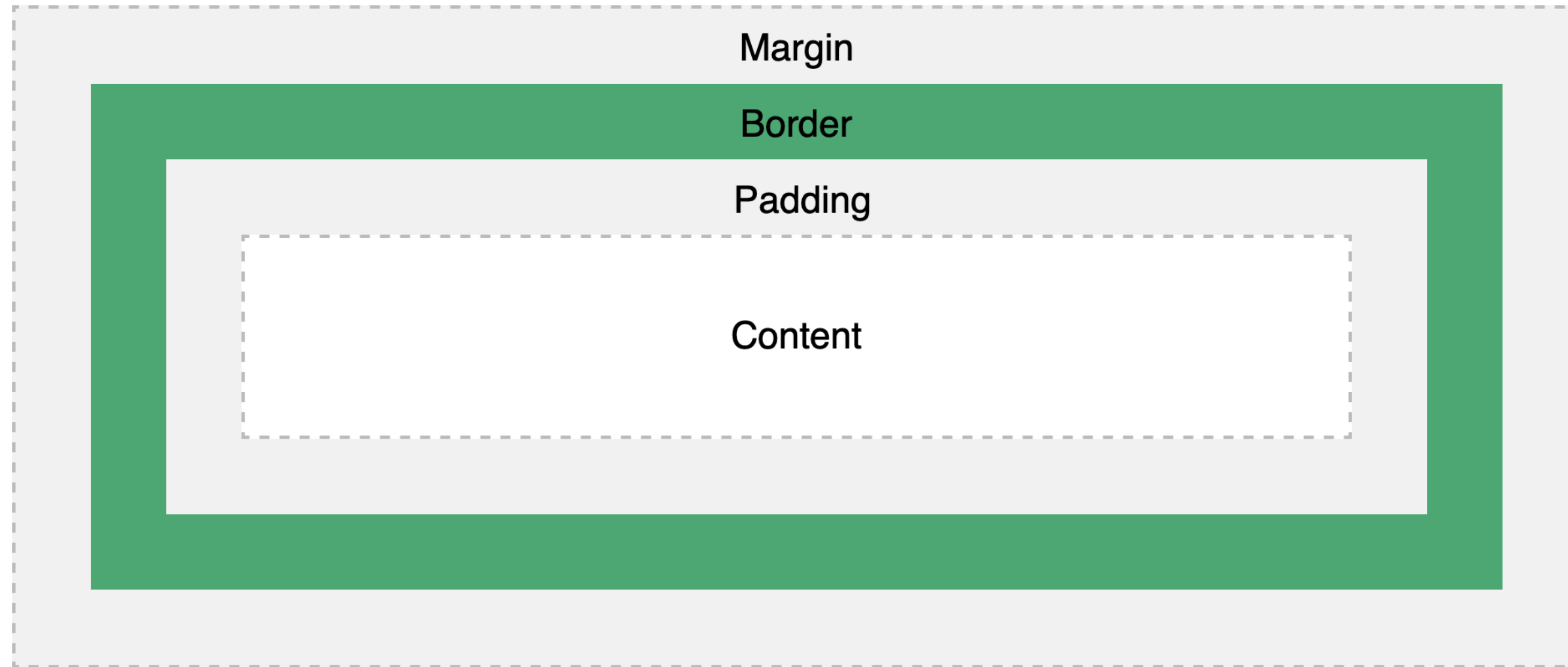
Introdução ao CSS

O Desafio

- Como saímos **disso...** para **isso?**

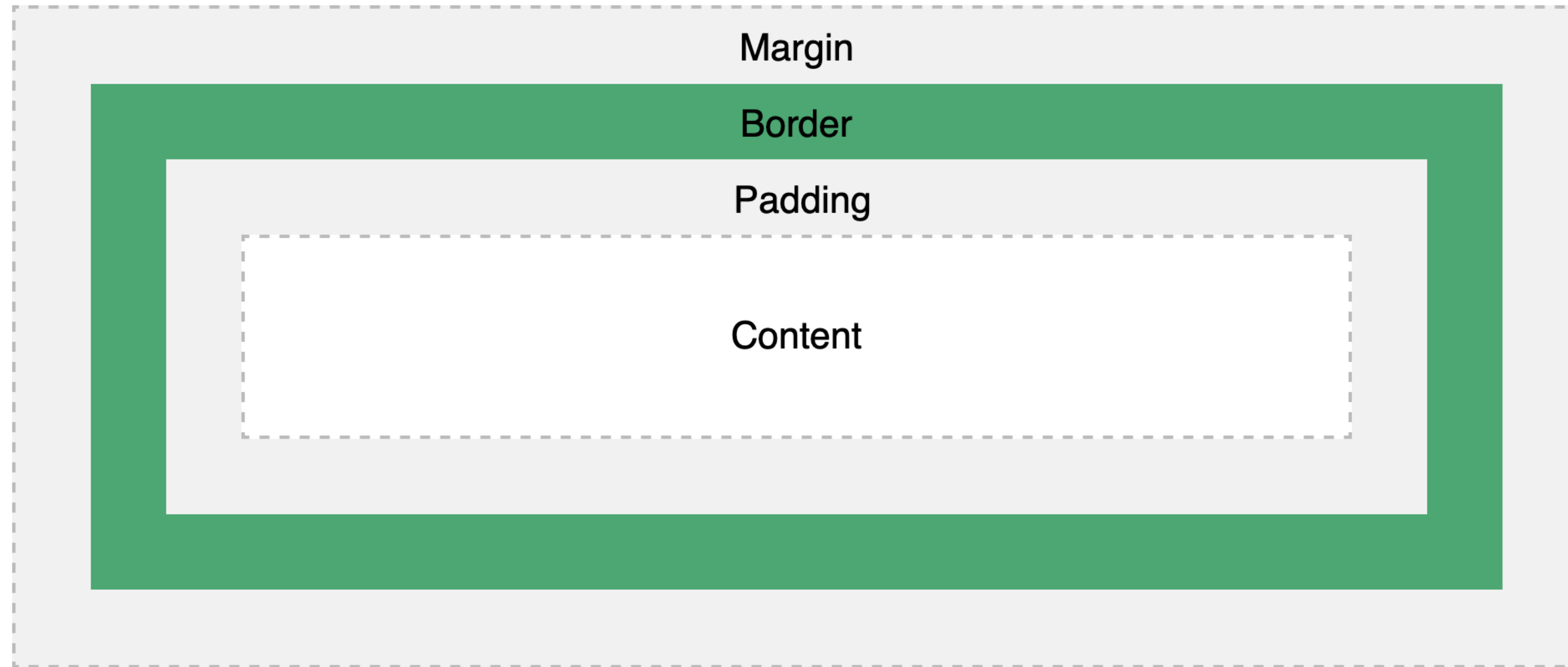


As 4 Camadas do Box Model



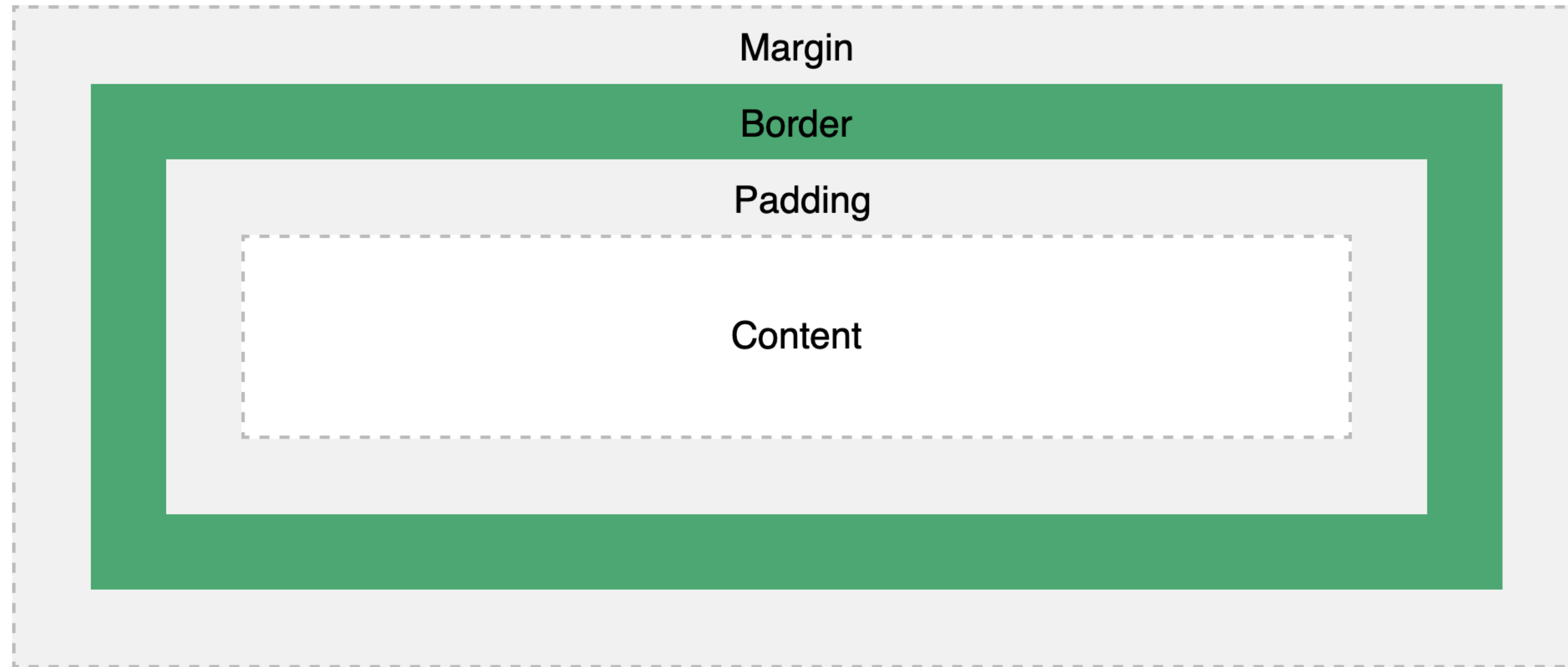
- **Content:** O conteúdo em si (texto, imagem).

As 4 Camadas do Box Model



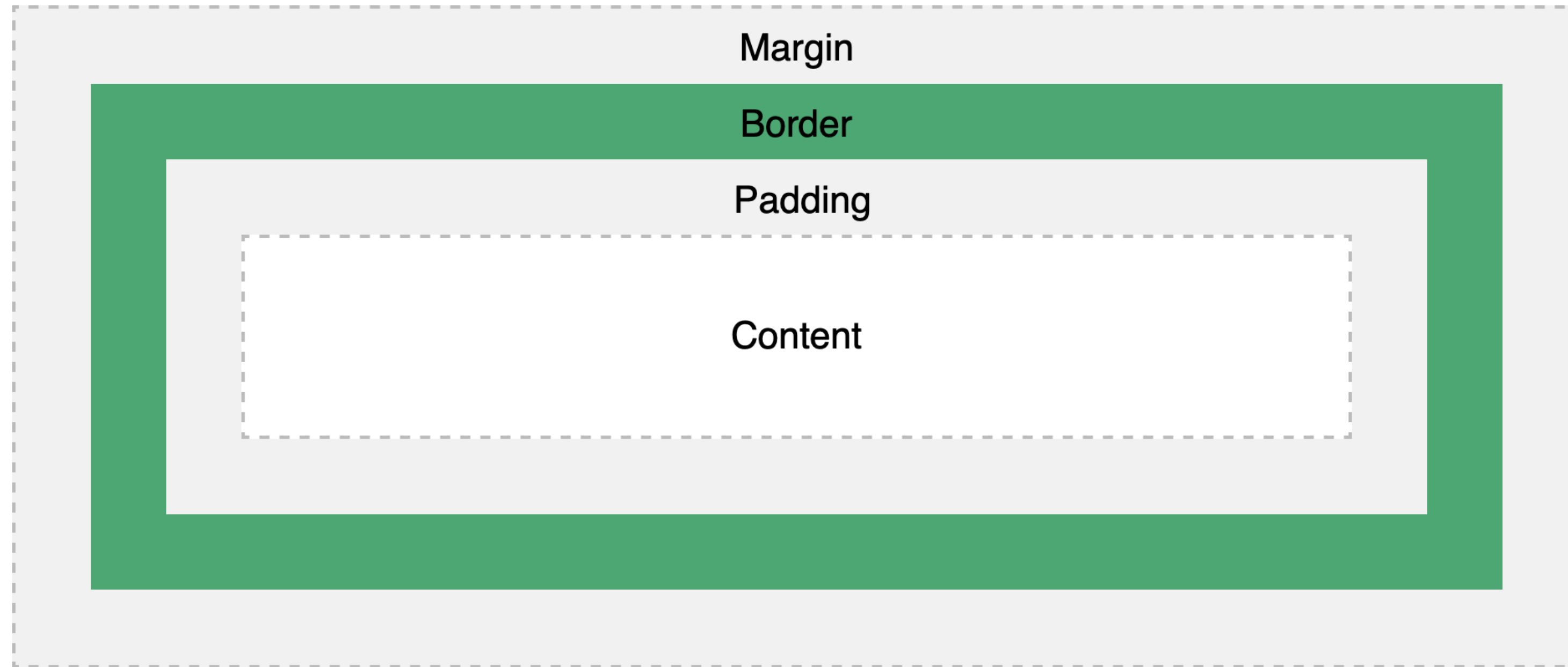
- **Padding:** O espaçamento interno. A distância entre o conteúdo e a parede da caixa.

As 4 Camadas do Box Model



- **Border:** A parede da caixa. A borda propriamente dita.

As 4 Camadas do Box Model



- **Margin:** O espaçamento externo. A "distância social" entre esta caixa e as caixas vizinhas.

O Problema do `box-sizing`

- **O Padrão Antigo** (`content-box`):
 - Imagine que você define que uma caixa tem `width: 200px;`.
 - Por padrão, o navegador aplica essa largura **apenas** ao conteúdo.
 - Se você adicionar `padding: 10px;` e `border: 1px;`, a largura total da caixa se torna $200 + 10 + 10 + 1 + 1 = 222\text{px}$.
 - Isso é um pesadelo para fazer cálculos de layout!

O Problema do `box-sizing`

- **A Solução Moderna** (`border-box`):
 - Com `box-sizing: border-box;`, a história muda.
 - Se você define `width: 200px;`, esta será a largura **final** da caixa, já incluindo o padding e a borda.
 - O navegador ajusta o espaço do conteúdo automaticamente. É muito mais intuitivo e é o padrão que usaremos.

Exercício Prático 1 - O Reset Universal

- Adicionar o seguinte código ao `style.css`:

```
* {  
    box-sizing: border-box;  
    margin: 0;  
    padding: 0;  
}  
  
body {  
    font-family: sans-serif; /* Recoloque aqui, pois o reset acima remove */  
}
```

- O `*` é um seletor universal. `margin: 0;` e `padding: 0;` removem os espaçamentos padrão do navegador, nos dando controle total.

O Conceito Central do Flexbox

- **Flex Container:** O elemento "pai" que irá organizar os filhos.
- **Flex Items:** Os elementos "filhos" diretos que serão organizados.
- **A Mágica começa com uma linha:** `display: flex;`

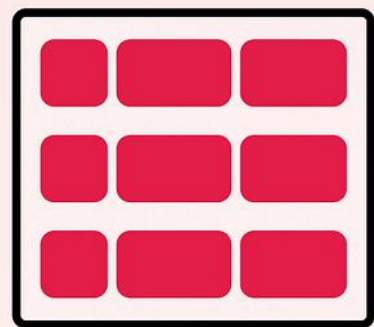
Propriedades do Flex Container

- A barra de navegação `<nav>`.
- **Problema:** Nossos links do menu estão um embaixo do outro. Queremos eles em uma linha horizontal.
- **Solução:**
 - No CSS, selecione o `<ul>` dentro da `nav`: `nav ul`.
 - Adicione `display: flex;`.
 - Adicione `list-style-type: none;` para remover as bolinhas da lista.

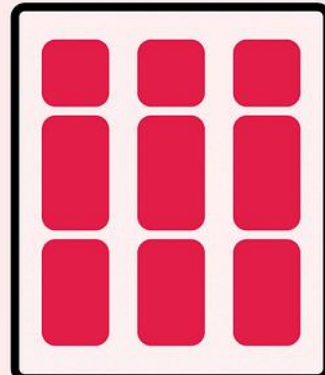
Propriedades do Flex Container

CSS Flexbox

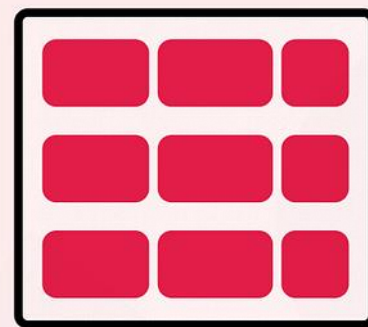
flex-direction



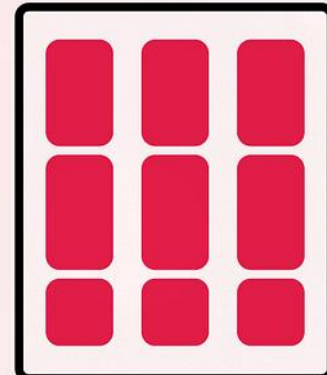
row



column

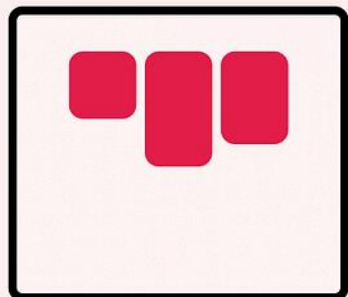


row-reverse

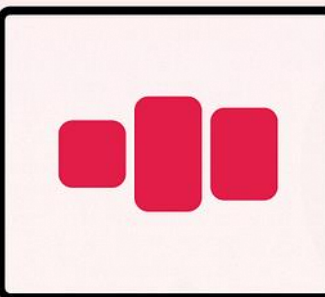


column-reverse

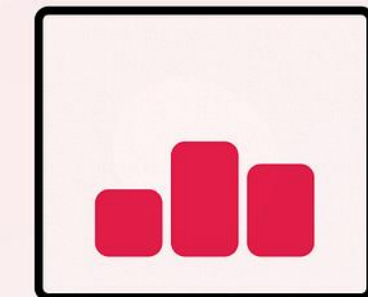
align-items



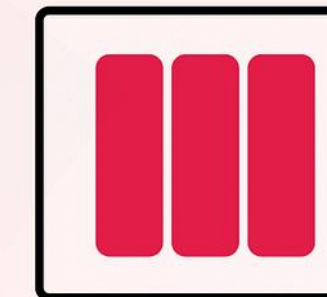
flex-start



center

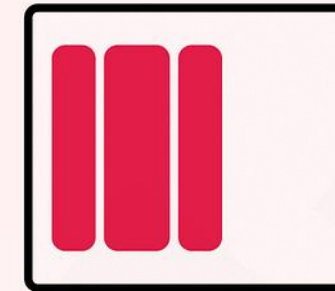


flex-end

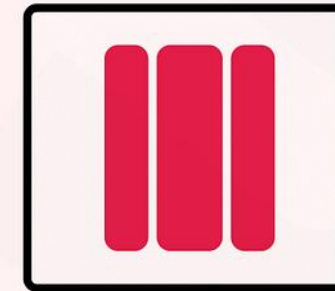


stretch

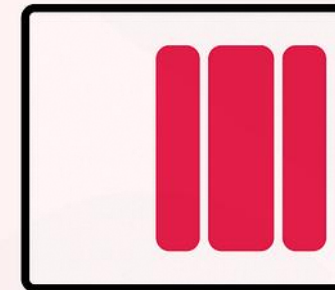
justify-content



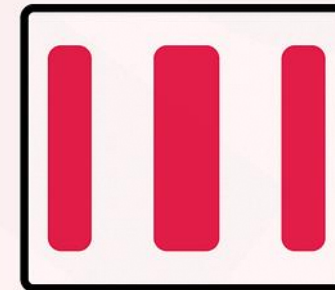
flex-start



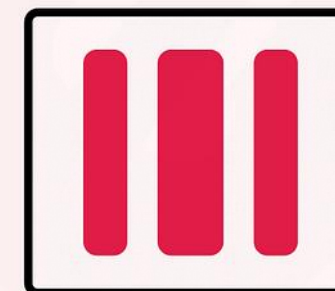
center



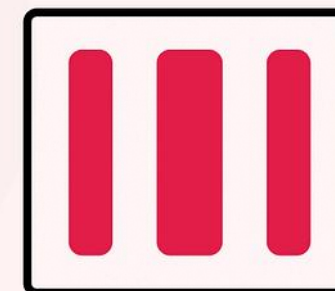
flex-end



space-between

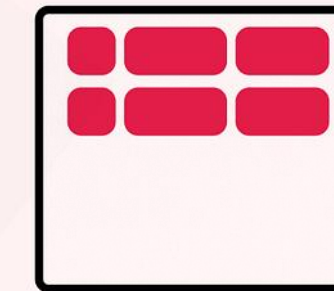


space-around

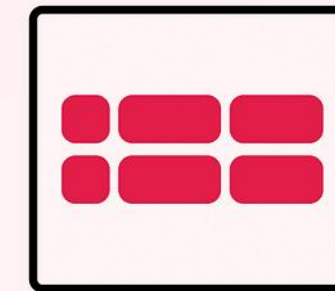


space-evenly

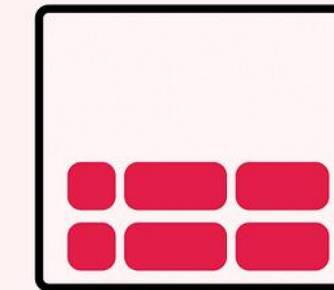
align-content



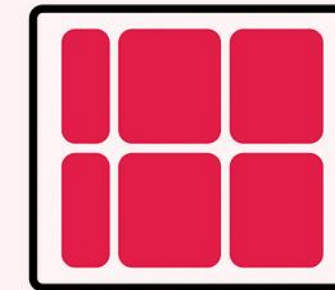
flex-start



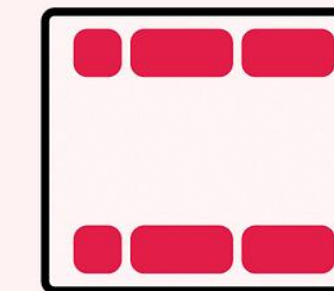
center



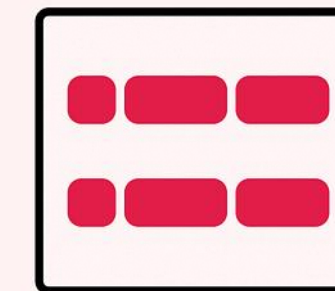
flex-end



stretch



space-between



space-around

Exercício Prático 2 - Criando a Barra de Navegação

- **Instruções:**

- Aplique `display: flex;`, `list-style-type: none;` no seu `nav ul`.
- Use `justify-content: flex-end;` para levar os links para a direita do header.
- Use `gap: 15px;` para dar um respiro entre os links.
- No header, adicione `display: flex;`, `justify-content: space-between;` e `align-items: center;` para alinhar o `<h1>` à esquerda e a `<nav>` à direita.

FlexBox Froogy

- **Instruções:** Faça até o nível 10 do jogo:

<https://flexboxfroggy.com/#pt-br>

Exercício Principal: Layout de Colunas

Preparando o Terreno (HTML)

- Primeiro, precisamos adicionar a barra lateral no nosso HTML. Dentro da tag `<main>`, logo após o fechamento do `<article>`, adicionem uma tag `<aside>`.

```
<main>
  <article class="post">
    </article>

  <aside class="sidebar">
    <h3>Posts Recentes</h3>
    <ul>
      <li><a href="#">Entendendo JavaScript</a></li>
      <li><a href="#">O que é React?</a></li>
    </ul>
  </aside>
</main>
```

Criando as Colunas com Flexbox

- **O Contêiner:** A tag `<main>` será nosso Flex Container.
- **Os Itens:** O `<article>` e o `<aside>` são nossos Flex Items.
- **Instruções para o `style.css`:**
 1. **Transforme o `<main>` em um contêiner flex:**

```
main {  
  display: flex;  
  gap: 30px; /* Espaço entre as colunas */  
  padding: 20px; /* Apenas para não colar nas bordas */  
}
```

Criando as Colunas com Flexbox

2. Defina os tamanhos das colunas.

Por padrão, eles dividem o espaço. Mas queremos que o artigo seja maior. Vamos usar a propriedade `flex` nos itens.

```
.post { /* A classe que já tínhamos no article */  
  flex: 2; /* Ocupará 2/3 do espaço */  
}  
.sidebar {  
  flex: 1; /* Ocupará 1/3 do espaço */  
}
```

Criando as Colunas com Flexbox

3. Adicione um estilo básico à sidebar para que ela se destaque.

Por padrão, eles dividem o espaço. Mas queremos que o artigo seja maior. Vamos usar a propriedade `flex` nos itens.

```
.sidebar {  
  background-color: #f4f4f4;  
  padding: 15px;  
  border-radius: 5px; /* Bordas arredondadas */  
}
```

Resumo e Conclusão

- **O que dominamos hoje:**
 - Todo elemento é uma caixa (**Box Model**).
 - `box-sizing: border-box` é nosso melhor amigo.
- **Flexbox** é a ferramenta moderna para criar layouts em uma dimensão (linhas ou colunas).
- Conseguimos criar um header e um layout de página com poucas linhas de código.

Próxima Aula

- **JavaScript: Fundamentos e DOM.**
- Nossa página agora tem estrutura (HTML) e estilo (CSS).
- Na próxima aula, vamos começar a dar inteligência e interatividade a ela com JavaScript!